

```

1 pipeline {
2
3     agent any
4
5     // -----
6     // Global environment variables
7     // -----
8     environment {
9         IMAGE_NAME      = 'flask-calculator'
10        CONTAINER_NAME   = 'flask-calculator-app'
11        APP_PORT         = '5000'
12        HOST_PORT        = '5000'
13        WORK_DIR         = 'project1'           // subfolder with Dockerfile
14
15        // — Docker Hub settings —————
16        DOCKERHUB_USER   = 'your-dockerhub-username' // ← change this
17        DOCKERHUB_REPO   = "${DOCKERHUB_USER}/${IMAGE_NAME}"
18        // Credential ID stored in Jenkins → Manage Credentials
19        DOCKERHUB_CREDS  = 'dockerhub-credentials' // ← match your cred ID
20    }
21
22    options {
23        timestamps()
24        buildDiscarder(logRotator(numToKeepStr: '10'))
25    }
26
27    stages {
28
29        // -----
30        // STAGE 1 – Clone the repository
31        // -----
32        stage('Checkout') {
33            steps {
34                echo '📄 Cloning repository...'
35                git branch: 'main',
36                    url: 'https://github.com/raj17rane/FLASK_PROJECTS.git'
37            }
38        }
39
40        // -----
41        // STAGE 2 – Build the Docker image
42        // -----
43        stage('Build Docker Image') {
44            steps {
45                echo '🐳 Building Docker image...'

```

```

46         dir("${WORK_DIR}") {
47             sh """
48                 docker build \
49                     -t ${IMAGE_NAME}:${BUILD_NUMBER} \
50                     -t ${IMAGE_NAME}:latest \
51                     .
52             """
53         }
54     }
55 }
56
57 // -----
58 // STAGE 3 – Push image to Docker Hub
59 // -----
60 stage('Push to Docker Hub') {
61     steps {
62         echo '🚀 Pushing image to Docker Hub...'
63         withCredentials([usernamePassword(
64             credentialsId: "${DOCKERHUB_CREDS}",
65             usernameVariable: 'DH_USER',
66             passwordVariable: 'DH_PASS'
67         )]) {
68             sh """
69                 echo "\$DH_PASS" | docker login -u "\$DH_USER" --password-stdin
70
71                 // Tag local image with Docker Hub repo name
72                 docker tag ${IMAGE_NAME}:${BUILD_NUMBER} ${DOCKERHUB_REPO}:${BUILD_NUMBER}
73                 docker tag ${IMAGE_NAME}:latest          ${DOCKERHUB_REPO}:latest
74
75                 // Push both tags
76                 docker push ${DOCKERHUB_REPO}:${BUILD_NUMBER}
77                 docker push ${DOCKERHUB_REPO}:latest
78
79                 docker logout
80             """
81         }
82     }
83 }
84
85 // -----
86 // STAGE 4 – Stop & remove any old running container
87 // -----
88 stage('Cleanup Old Container') {
89     steps {
90         echo '🧹 Removing old container if it exists...'

```

```

91         sh """
92             docker stop ${CONTAINER_NAME} || true
93             docker rm    ${CONTAINER_NAME} || true
94         """
95     }
96 }
97
98 // -----
99 // STAGE 5 – Deploy on localhost (same server as Jenkins)
100 // Pulls the freshly pushed image from Docker Hub and runs it
101 // -----
102 stage('Deploy to Localhost') {
103     steps {
104         echo '🚀 Deploying Flask app on localhost...'
105         withCredentials([usernamePassword(
106             credentialsId: "${DOCKERHUB_CREDS}",
107             usernameVariable: 'DH_USER',
108             passwordVariable: 'DH_PASS'
109         )]) {
110             sh """
111                 // Pull the latest image from Docker Hub to confirm registry sync
112                 echo "\$DH_PASS" | docker login -u "\$DH_USER" --password-stdin
113                 docker pull ${DOCKERHUB_REPO}:latest
114                 docker logout
115
116                 // Run the container on this host
117                 docker run -d \
118                     --name  ${CONTAINER_NAME} \
119                     -p      ${HOST_PORT}:${APP_PORT} \
120                     --restart unless-stopped \
121                     ${DOCKERHUB_REPO}:latest
122             """
123         }
124     }
125 }
126
127 // -----
128 // STAGE 6 – Health check / Smoke test
129 // -----
130 stage('Smoke Test') {
131     steps {
132         echo '🔍 Waiting for app to be ready...'
133         sh """
134             for i in \$(seq 1 30); do
135                 if curl -sf http://localhost:${HOST_PORT}/ > /dev/null; then

```

```

136         echo '✅ App is up and healthy!'
137         exit 0
138     fi
139     echo " attempt \$i – not ready yet, sleeping 2s..."
140     sleep 2
141 done
142 echo '❌ App did not start within 60 seconds.'
143 docker logs ${CONTAINER_NAME}
144 exit 1
145 """"
146
147 // Assert HTTP 200
148 sh """"
149     STATUS=$(curl -o /dev/null -s -w "%{http_code}" \
150         http://localhost:${HOST_PORT}/)
151     echo "HTTP response status: \$STATUS"
152     if [ "\$STATUS" != "200" ]; then
153         echo "❌ Expected HTTP 200, got \$STATUS"
154         exit 1
155     fi
156     echo "✅ Smoke test passed – HTTP 200 OK"
157 """"
158 }
159 }
160
161 } // end stages
162
163 // -----
164 // Post-build actions
165 // -----
166 post {
167
168     success {
169         echo """"
170         =====
171         ✅ BUILD SUCCESSFUL
172         Image  : ${DOCKERHUB_REPO}:${BUILD_NUMBER}
173         App    : http://localhost:${HOST_PORT}/
174         =====
175         """"
176     }
177
178     failure {
179         echo '❌ Build failed – printing container logs for diagnosis...'
180         sh "docker logs ${CONTAINER_NAME} || true"

```

```
181         }
182
183     always {
184         echo '📋 Pipeline finished.'
185     }
186
187 }
188
189 }
190
```